Unit 4 - Recurrent neural networks (RNNs)

4.1. Sequence data

4.1.1. Time series data

- A timeseries can be any data obtained via measurements at regular intervals, like the daily price of a stock, the hourly electricity consumption of a city, or the weekly sales of a store.

- **Time series related tasks**:

  - **Forecasting**: By far, the most common task with time series. Consists of predicting what will happen next in a series:
    - Forecast electricity consumption a few hours in advance so you can anticipate demand.
    - Forecast the weather a few days in advance so you can plan your schedule.
  - **Classification**: Assign one or more categorical labels to a timeseries.
    - For instance, given the timeseries of the activity of a visitor on a website, classify whether the visitor is a bot or a human.
  - **Event detection**: Identify the occurrence of a specific expected event within a continuous data stream.
    - Identifying the different phases of sleep in polysomnographic data.
    - Identify a sleep apnea event analysing the signals for respiratory flow, ECG, EOG, EMG, etc.
  - **Anomaly detection**: Detect anything unusual happening within a continuous datastream.
    - Detecting unusual activity on the corporate network, might be an attacker.
    - Anomaly detection is typically done via unsupervised learning, because you often don't know what kind of anomaly you're looking for, so you can't train on specific anomaly examples.

4.1.2. A temperature-forecasting example (Jena dataset)

- **Inspecting the data of the Jena weather dataset**

  - We'll work with a weather timeseries dataset recorded at the weather station at the Max Planck Institute for Biogeochemistry in Jena, Germany (www.bgc-jena.mpg.de/wetter).
  - In this dataset, **14 different quantities** (such as temperature, pressure, humidity, wind direction, and so on) were recorded every 10 minutes over several years.

- The original data goes back to 2003, but the subset of the data we'll download is limited to 2009–2016.
- Let's look at the data.

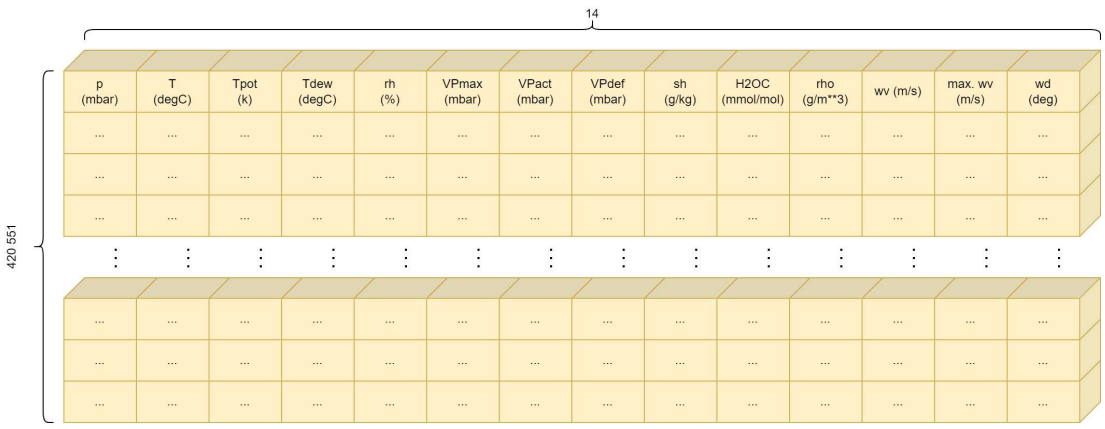
In [1]:
```
import os
# The code assumes that the CSV file is in this location
fname = os.path.join("datasets/jena_climate_2009_2016/jena_climate_2009_2016.csv

with open(fname) as f:
    data = f.read()

lines = data.split("\n")
header = lines[0].split(",")
lines = lines[1:]
print(header)
print(len(lines))
```

```
['"Date Time"', '"p (mbar)"', '"T (degC)"', '"Tpot (K)"', '"Tdew (degC)"', '"rh
(%)"', '"VPmax (mbar)"', '"VPact (mbar)"', '"VPdef (mbar)"', '"sh (g/kg)"', '"H2O
C (mmol/mol)"', '"rho (g/m**3)"', '"wv (m/s)"', '"max. wv (m/s)"', '"wd (deg)"']
420451
```

- **Parsing the data**
  - We convert all 420,551 lines of data into NumPy arrays: one array for the temperature (in degrees Celsius), and another one for the rest of the data—the features we will use to predict future temperatures.
  - Note that we discard the "Date Time" column.

In [2]:
```
import numpy as np
temperature = np.zeros((len(lines),))
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1] # Column 1 is the temperature array
    raw_data[i, :] = values[:] # All columns (including temperature) is the "raw
```

- **`raw_data` representation**



In [3]:
```
print(temperature)
```

```
[-8.02 -8.41 -8.51 ... -3.16 -4.23 -4.82]
```

In [4]:
```
temperature.shape
```

Out[4]:
```
(420451,)
```

In [5]:
```
print(raw_data)
```

```
[[ 9.9652e+02 -8.0200e+00  2.6540e+02 ...  1.0300e+00  1.7500e+00
   1.5230e+02]
 [ 9.9657e+02 -8.4100e+00  2.6501e+02 ...  7.2000e-01  1.5000e+00
   1.3610e+02]
 [ 9.9653e+02 -8.5100e+00  2.6491e+02 ...  1.9000e-01  6.3000e-01
   1.7160e+02]
 ...
 [ 9.9982e+02 -3.1600e+00  2.7001e+02 ...  1.0800e+00  2.0000e+00
   2.1520e+02]
 [ 9.9981e+02 -4.2300e+00  2.6894e+02 ...  1.4900e+00  2.1600e+00
   2.2580e+02]
 [ 9.9982e+02 -4.8200e+00  2.6836e+02 ...  1.2300e+00  1.9600e+00
   1.8490e+02]]
```

In [6]:
```
raw_data.shape
```

Out[6]:
```
(420451, 14)
```

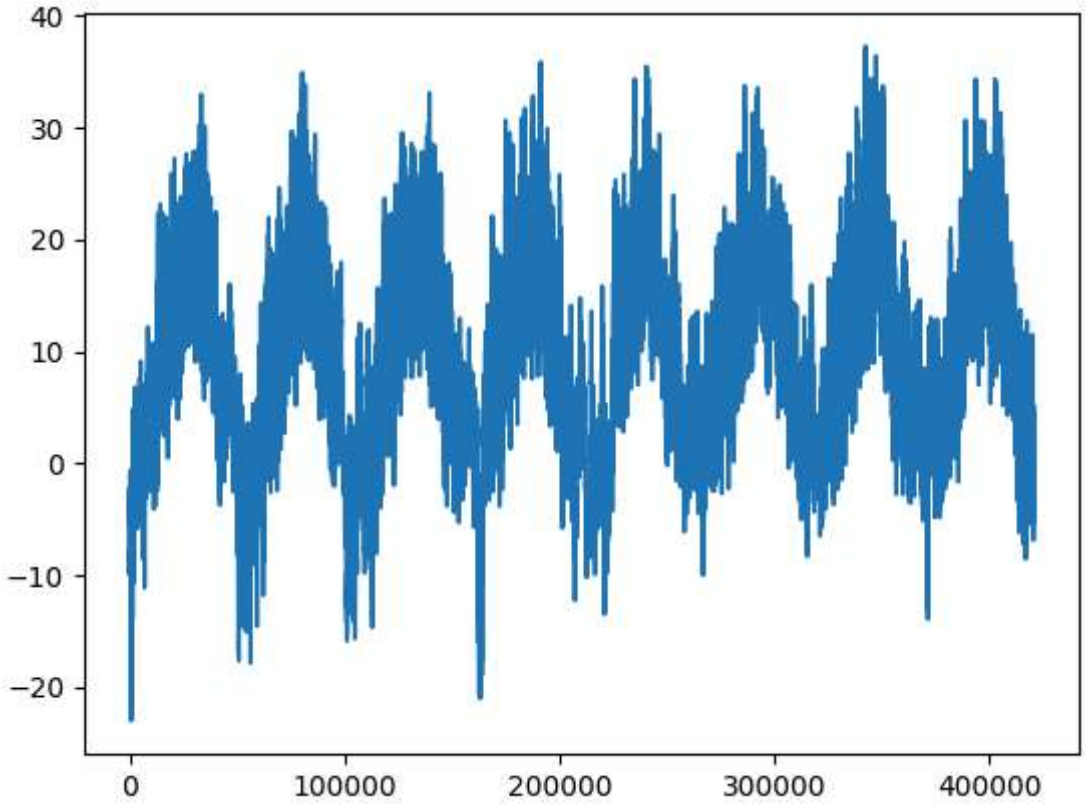
In [7]:
```
raw_data[:, 0:1].shape
```

Out[7]:
```
(420451, 1)
```

- **Plotting the temperature timeseries**
  - In this plot we can see the temperature (in degrees Celsius) over time.
  - We can clearly see the yearly periodicity of temperature—the data spans 8 years.

In [8]:
```
from matplotlib import pyplot as plt
plt.plot(range(len(temperature)), temperature)
```

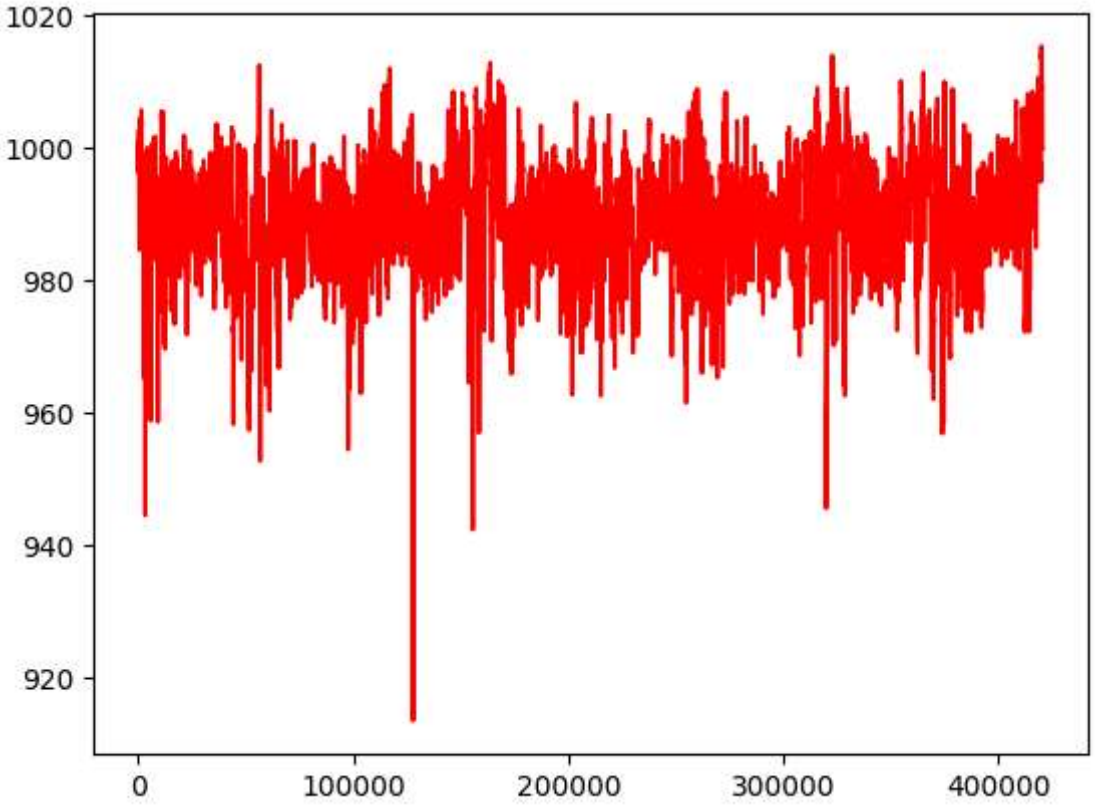
Out[8]:
```
[<matplotlib.lines.Line2D at 0x2d87e1436e0>]
```



- **Plotting other quantities**

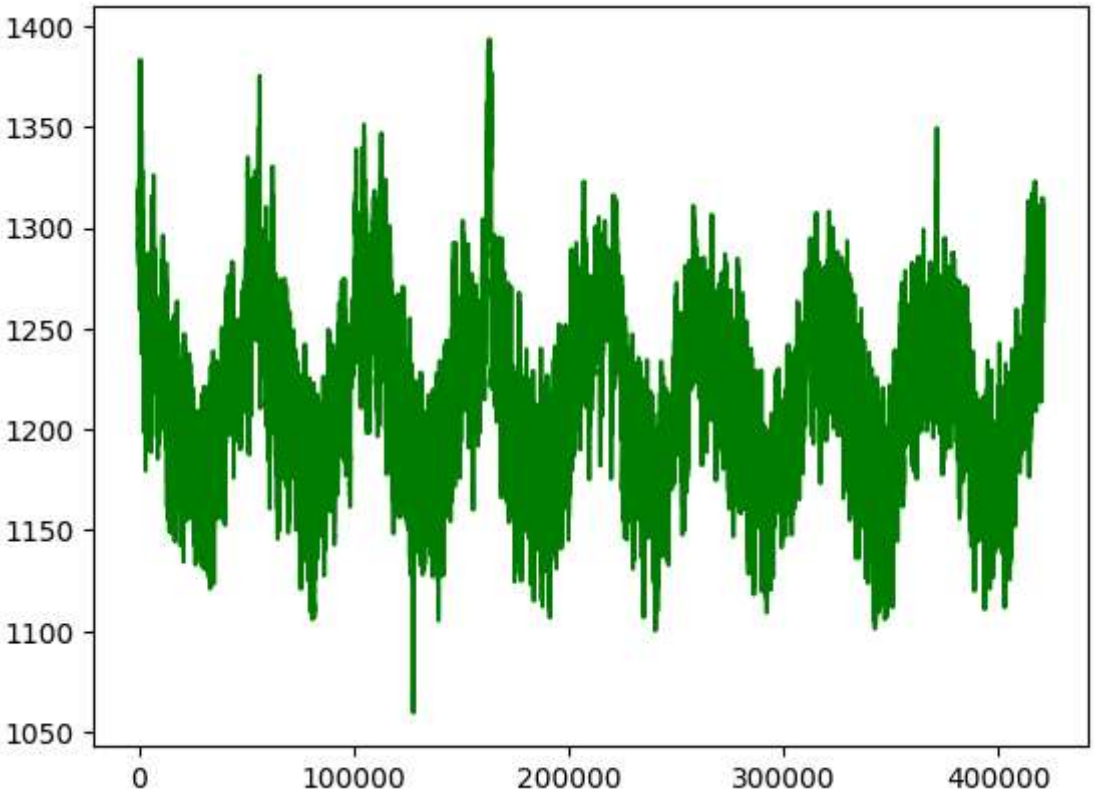
In [9]:
```
# Plotting the pression in mbar

from matplotlib import pyplot as plt
plt.plot(range(len(raw_data[:, 0:1])), raw_data[:, 0:1], "r")

# raw_data[:, 0:1] is a NumPy array slice that takes all the rows
# and the columns starting in zero and finishing before 1
```

Out[9]:
```
[<matplotlib.lines.Line2D at 0x2d87f129f70>]
```



In [10]:
```
# Plotting the density (rho) in g/m**3

from matplotlib import pyplot as plt
plt.plot(range(len(raw_data[:, 10:11])), raw_data[:, 10:11], "g")
```

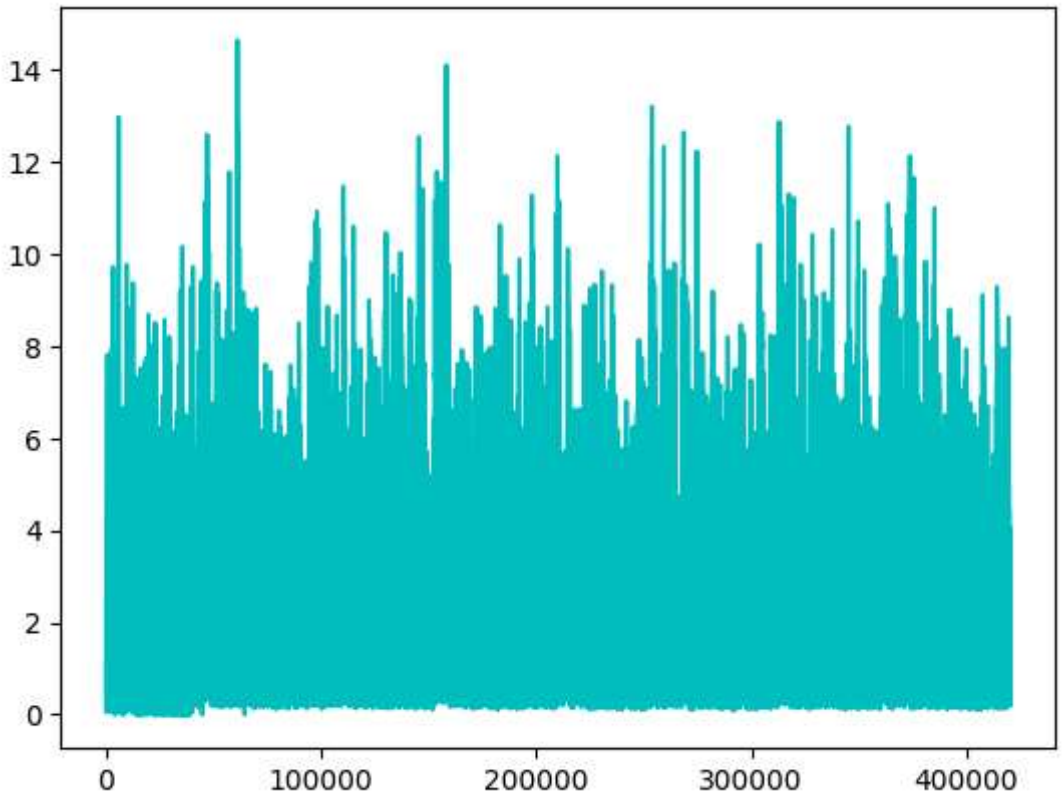
Out[10]: [<matplotlib.lines.Line2D at 0x2d87e189700>]



In [11]:
```
# Plotting the wind velocity in m/s
```

```
from matplotlib import pyplot as plt
plt.plot(range(len(raw_data[:, 11:12])), raw_data[:, 11:12], "c")
```

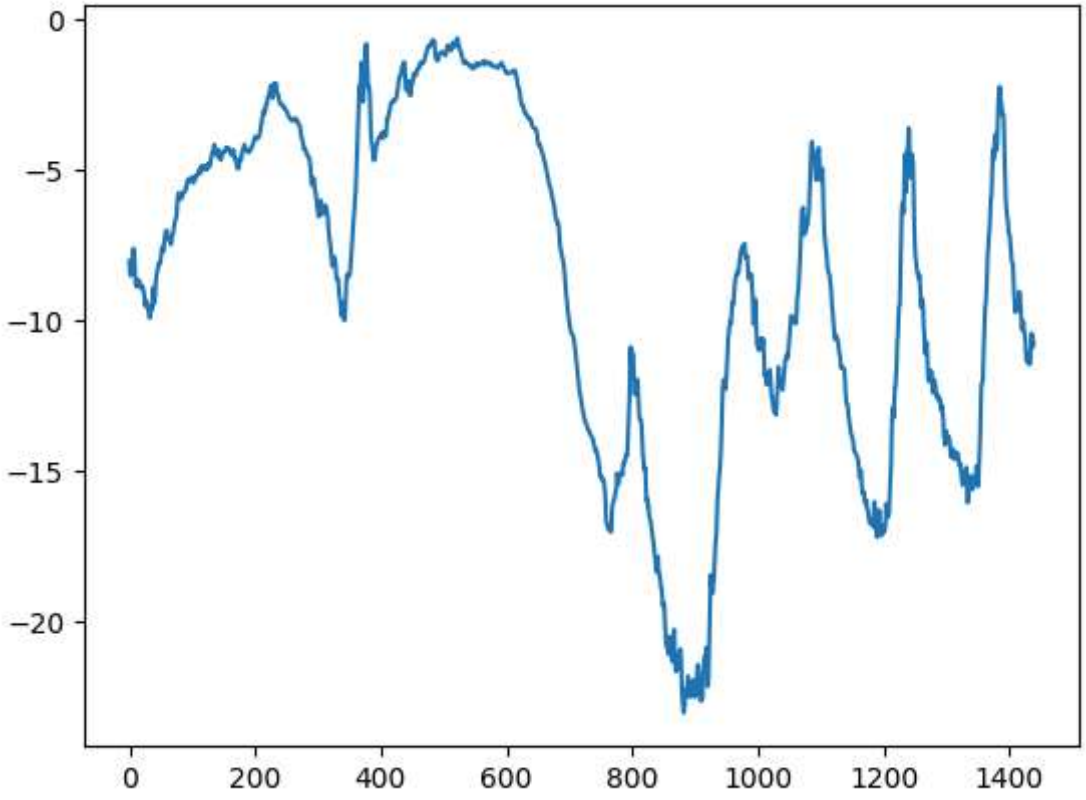
Out[11]: [<matplotlib.lines.Line2D at 0x2d87f209580>]



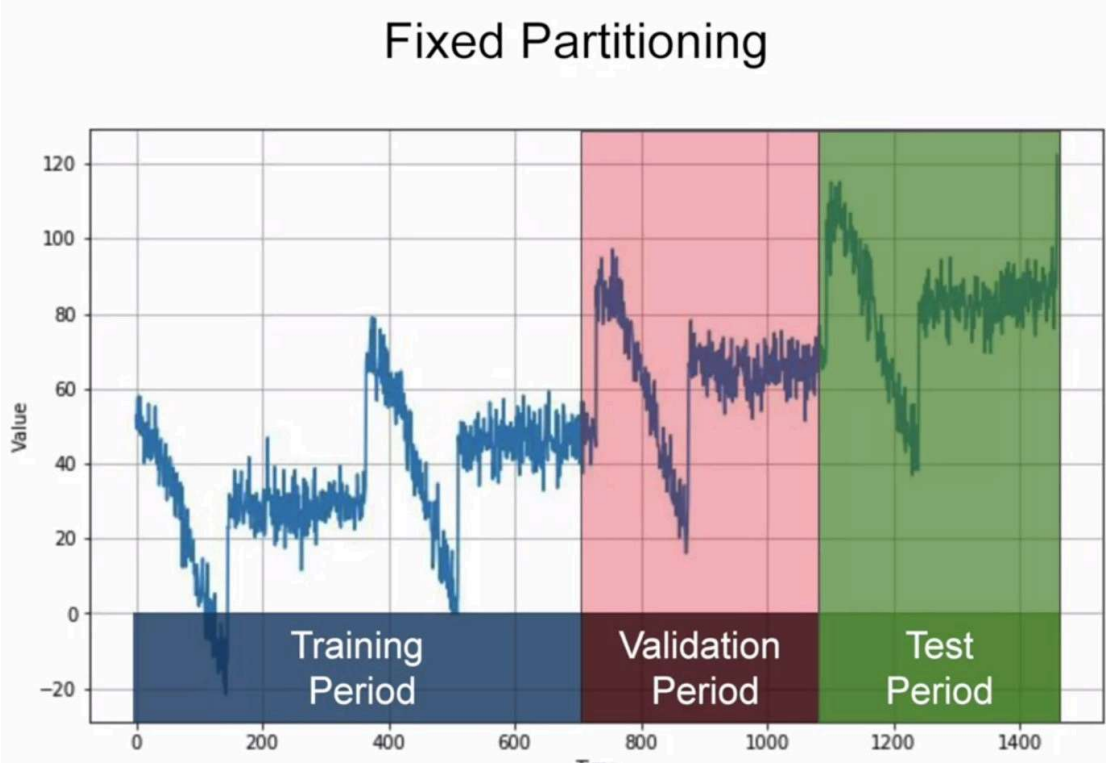
- **Plotting the first 10 days of the temperature timeseries**
  - We can see a more narrow plot of the first 10 days of temperature data.
  - Because the data is recorded every 10 minutes, you get 24 × 6 = 144 data points per day.

In [12]:
```
plt.plot(range(1440), temperature[:1440])
```

Out[12]: [<matplotlib.lines.Line2D at 0x2d8075bf4a0>]



- **Computing the number of samples we'll use for each data split**
  - We'll use the first 50% of the data for training, the following 25% for validation, and the last 25% for testing.
  - When working with timeseries data, it's important to use validation and test data that is more recent than the training data, because we're trying to predict the future given the past, not the reverse, and your validation/test splits should reflect that.



In [13]:
```
len(raw_data)
```

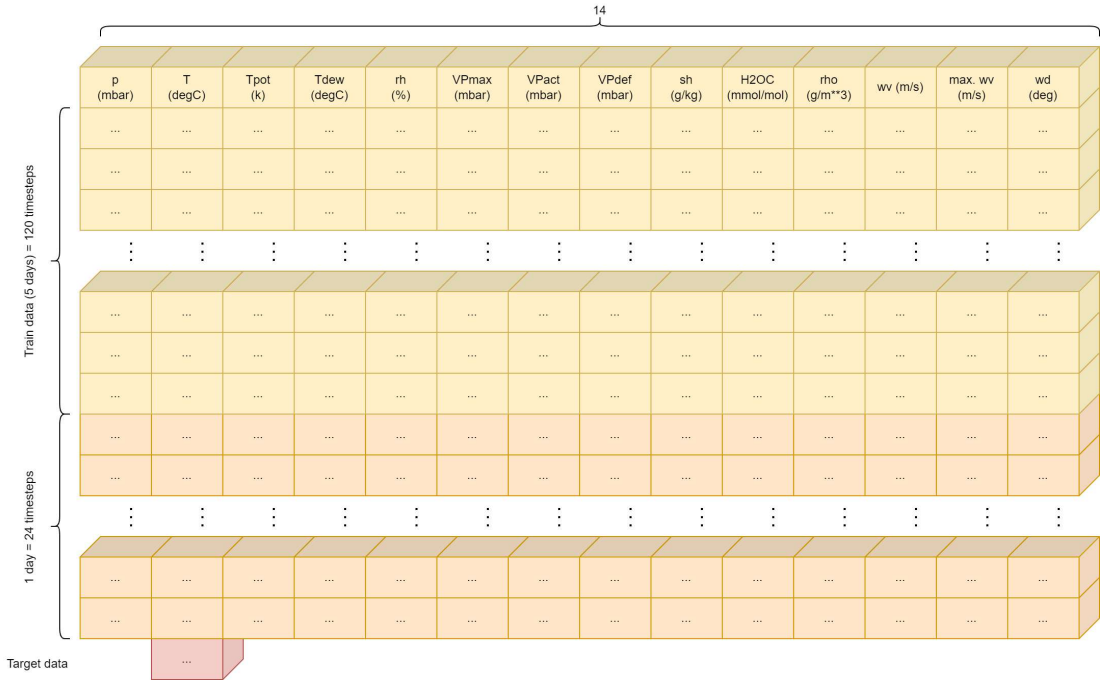
Out[13]:
```
420451
```

In [14]:
```
num_train_samples = int(0.5 * len(raw_data))
num_val_samples = int(0.25 * len(raw_data))
num_test_samples = len(raw_data) - num_train_samples - num_val_samples
print("num_train_samples:", num_train_samples)
print("num_val_samples:", num_val_samples)
print("num_test_samples:", num_test_samples)
```

```
num_train_samples: 210225
num_val_samples: 105112
num_test_samples: 105114
```

4.1.3. Problem to solve with the Jena dataset

- The exact formulation of the problem will be as follows: **given data covering the previous five days and sampled once per hour, can we predict the temperature in 24 hours?**



- **Normalizing the data**
  - Let's preprocess the data to a format a neural network can ingest.
  - Each timeseries in the data is on a different scale (for example, atmospheric pressure, measured in mbar, is around 1,000, while H2OC, measured in millimoles per mole, is around 3).
  - We'll normalize each timeseries independently so that they all take small values on a similar scale.
  - We're going to use the first 210,225 timesteps as training data, so we'll compute the mean and standard deviation only on this fraction of the data.

In [15]:
```
print(raw_data)
```

```
[[ 9.9652e+02 -8.0200e+00  2.6540e+02 ...  1.0300e+00  1.7500e+00
   1.5230e+02]
 [ 9.9657e+02 -8.4100e+00  2.6501e+02 ...  7.2000e-01  1.5000e+00
   1.3610e+02]
 [ 9.9653e+02 -8.5100e+00  2.6491e+02 ...  1.9000e-01  6.3000e-01
   1.7160e+02]
 ...
 [ 9.9982e+02 -3.1600e+00  2.7001e+02 ...  1.0800e+00  2.0000e+00
   2.1520e+02]
 [ 9.9981e+02 -4.2300e+00  2.6894e+02 ...  1.4900e+00  2.1600e+00
   2.2580e+02]
 [ 9.9982e+02 -4.8200e+00  2.6836e+02 ...  1.2300e+00  1.9600e+00
   1.8490e+02]]
```

- We will use the reduction methods of NumPy `mean` and `std`.
- Reduction methods accept the `axis` to apply the reduction on.
- In our case we want the mean and standard deviation of each column.

In [16]:
```
# Example
b = np.arange(12).reshape(4,3) # 2d array
```

```
print(b)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

In [17]:
```
b.mean()
```

Out[17]:
```
5.5
```

In [18]:
```
b.mean(axis=0)
```

Out[18]:
```
array([4.5, 5.5, 6.5])
```

In [19]:
```
b.mean(axis=1)
```

Out[19]:
```
array([ 1.,  4.,  7., 10.])
```

In [20]:
```
mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean

std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

In [21]:
```
print(raw_data)
```

```
[[ 0.91365151 -1.92064015 -1.97449272 ... -0.73016651 -0.77935289
  -0.28119316]
 [ 0.91953033 -1.96510495 -2.01848295 ... -0.93230685 -0.88696976
  -0.46989368]
 [ 0.91482727 -1.97650618 -2.0297625  ... -1.27790162 -1.26147647
  -0.05638329]
 ...
 [ 1.30165361 -1.36654038 -1.45450563 ... -0.69756323 -0.67173602
   0.45147737]
 [ 1.30047784 -1.48853354 -1.57519677 ... -0.43021634 -0.60286122
   0.57494808]
 [ 1.30165361 -1.5558008  -1.64061814 ... -0.59975339 -0.68895472
   0.09853751]]
```

- **Dataset object creation with `timeseries_dataset_from_array()`**
  - Next, let's create a `Dataset` object that yields batches of data from the past five days along with a target temperature 24 hours in the future.
  - Because the samples in the dataset are highly redundant (sample N and sample N + 1 will have most of their timesteps in common), it would be wasteful to explicitly allocate memory for every sample.
  - Instead, we'll generate the samples on the fly while only keeping in memory the original `raw_data` and `temperature` arrays, and nothing more.
  - We could easily write a Python generator to do this, but there's a built-in dataset utility in Keras that does just that (`timeseries_dataset_from_array()`), so we can save ourselves some work by using it: https://keras.io/api/preprocessing/timeseries/

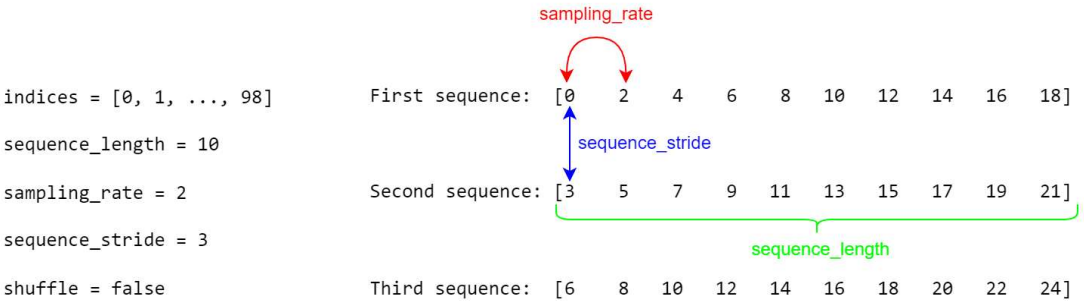
- **Arguments of `timeseries_dataset_from_array()`**
  - `data` : Numpy array or eager tensor containing consecutive data points (timesteps). Axis 0 is expected to be the time dimension.
  - `targets` : Targets corresponding to timesteps in data. `targets[i]` should be the target corresponding to the window that starts at index `i` . Pass `None` if you don't have target data (in this case the dataset will only yield the input data).
  - `sequence_length` : Length of the output sequences (in number of timesteps).
  - `sequence_stride` : Period between successive output sequences. For stride `s` , output samples would start at index `data[i], data[i + s], data[i + 2 * s]` , etc.
  - `sampling_rate` : Period between successive individual timesteps within sequences. For rate `r` , timesteps `data[i], data[i + r], ... data[i + sequence_length]` are used for create a sample sequence.
  - `batch_size` : Number of timeseries samples in each batch (except maybe the last one).
  - `shuffle` : Whether to shuffle output samples, or instead draw them in chronological order.
  - `seed` : Optional int; random seed for shuffling.
  - `start_index` : Optional int; data points earlier (exclusive) than `start_index` will not be used in the output sequences. This is useful to reserve part of the data for test or validation.
  - `end_index` : Optional int; data points later (exclusive) than `end_index` will not be used in the output sequences. This is useful to reserve part of the data for test or validation.

- **Example of `timeseries_dataset_from_array()` function:**

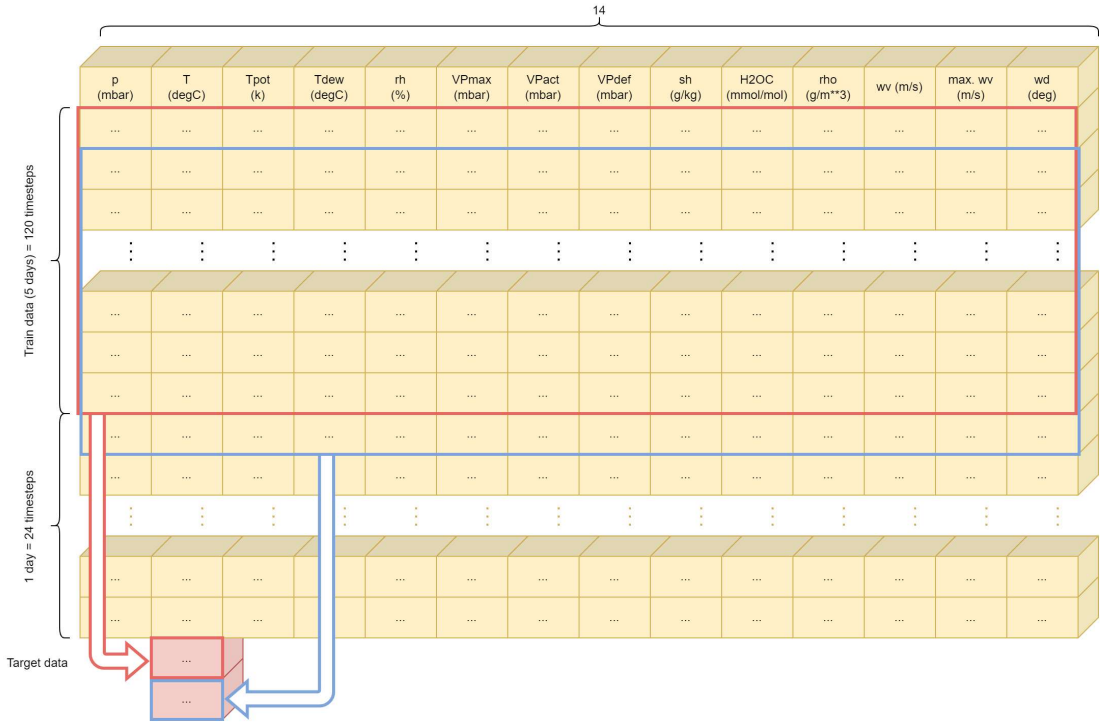
In [22]:
```
import numpy as np
import keras
int_sequence = np.arange(10)  # Our raw data [0 1 2 3 4 5 6 7 8 9]
dummy_dataset = keras.utils.timeseries_dataset_from_array(
    data=int_sequence[:-3],   # it will be [0 1 2 3 4 5 6] excluding the last th
    targets=int_sequence[3:], # Target for the sequence that starts at data[N] w
    sequence_length=3,        # The sequences will be 3 steps long: [0 1 2], [1
    batch_size=2,             # The sequences will be batched in batches of size
)

for inputs, targets in dummy_dataset:
    for i in range(inputs.shape[0]):
        print([int(x) for x in inputs[i]], int(targets[i]))
```

```
[0, 1, 2] 3
[1, 2, 3] 4
[2, 3, 4] 5
[3, 4, 5] 6
[4, 5, 6] 7
```

- **Difference between `sequence_length`, `sequence_stride` and `sampling_rate`:**



- **Instantiating datasets for training, validation, and testing**
  - We'll use `timeseries_dataset_from_array()` to instantiate three datasets: one for training, one for validation, and one for testing.
  - We'll use the following parameter values:
    - `sampling_rate = 6`: Observations will be sampled at one data point per hour: we will only keep one data point out of 6.
    - `sequence_length = 120`: Observations will go back 5 days (120 hours).
    - `delay = sampling_rate * (sequence_length + 24 - 1)`: The target for a sequence will be the temperature 24 hours after the end of the sequence.
  - Datasets indexes:
    - Training dataset: `start_index = 0` and `end_index = num_train_samples` to only use the first 50% of the data.
    - Validation dataset: `start_index = num_train_samples` and `end_index = num_train_samples + num_val_samples` to use the next 25% of the data.
    - Test dataset: `start_index = num_train_samples + num_val_samples` to use the remaining samples.



In [23]:
```
sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets         = temperature[delay:],
    sampling_rate   = sampling_rate,
    sequence_length = sequence_length,
    shuffle         = True,
    batch_size      = batch_size,
    start_index     = 0,
    end_index       = num_train_samples)

val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets         = temperature[delay:],
    sampling_rate   = sampling_rate,
    sequence_length = sequence_length,
    shuffle         = True,
    batch_size      = batch_size,
    start_index     = num_train_samples,
    end_index       = num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets         = temperature[delay:],
    sampling_rate   = sampling_rate,
    sequence_length = sequence_length,
    shuffle         = True,
    batch_size      = batch_size,
    start_index     = num_train_samples + num_val_samples)
```

- **Inspecting the output of one of our datasets**

- Each dataset yields a tuple `(samples, targets)`
  - `samples` is a batch of 256 samples, each containing 120 consecutive hours of input data (14 variables)
  - `targets` is the corresponding array of 256 target temperatures.
- Note that the samples are randomly shuffled, so two consecutive sequences in a batch (like `samples[0]` and `samples[1]`) aren't necessarily temporally close.

In [24]:
```
for samples, targets in train_dataset:
    print("samples shape:", samples.shape)
    print("targets shape:", targets.shape)
    break
```

```
samples shape: (256, 120, 14)
targets shape: (256,)
```

4.2. Using sequence data without recurrence

4.2.1. A common-sense, non-machine-learning baseline

- **Computing the common-sense baseline MAE**
  - Before we start using black-box deep learning models to solve the temperature-prediction problem, let's try a simple, common-sense approach.
  - In this case, the temperature timeseries can safely be assumed to be continuous (the temperatures tomorrow are likely to be close to the temperatures today) as well as periodical with a daily period.
  - Thus a common-sense approach is to always predict that the temperature 24 hours from now will be equal to the temperature right now. Let's evaluate this approach, using the mean absolute error (MAE) metric, defined as follows: `np.mean(np.abs(preds - targets))`

In [25]:
```
def evaluate_naive_method(dataset):
    total_abs_err = 0.
    samples_seen = 0
    for samples, targets in dataset:
        # samples[:, -1, 1] is the last temperature measurement in the input seq
        # Data is un-normalized by multiplying it by the sd and adding back the
        preds = samples[:, -1, 1] * std[1] + mean[1]
        total_abs_err += np.sum(np.abs(preds - targets))
        samples_seen += samples.shape[0]
    return total_abs_err / samples_seen

print(f"Validation MAE: {evaluate_naive_method(val_dataset):.2f}")
print(f"Test MAE: {evaluate_naive_method(test_dataset):.2f}")
```

```
Validation MAE: 2.44
Test MAE: 2.62
```

- if we always assume that the temperature 24 hours in the future will be the same as it is now, we will be off by two and a half degrees on average.

- It's not too bad, but we probably won't launch a weather forecasting service based on this heuristic.
- Let's use deep learning to do it better.

4.2.2. Time series data with dense networks

Training and evaluating a densely connected model

- In the same way that it's useful to establish a common-sense baseline before trying machine learning approaches, it's useful to try simple, cheap machine learning models (such as small, densely connected networks) before looking into complicated and computationally expensive models such as RNNs.
- This is the best way to make sure any further complexity you throw at the problem is legitimate and delivers real benefits.
- The following listing shows a fully connected model that starts by flattening the data and then runs it through two `Dense` layers.
- Note the lack of an activation function on the last `Dense` layer, which is typical for a regression problem.
- We use mean squared error (MSE) as the loss, rather than MAE, because unlike MAE, it's smooth around zero, which is a useful property for gradient descent. We will monitor MAE by adding it as a metric in `compile()`.

In [27]:
```
# Note: We call "tf.config.run_functions_eagerly(True)" to enable eager executio
# This avoids the construction of a execution graph, something that can be usefu
# Included here to avoid an unexpected "Graph execution error".

import tensorflow as tf
tf.config.run_functions_eagerly(True)
```

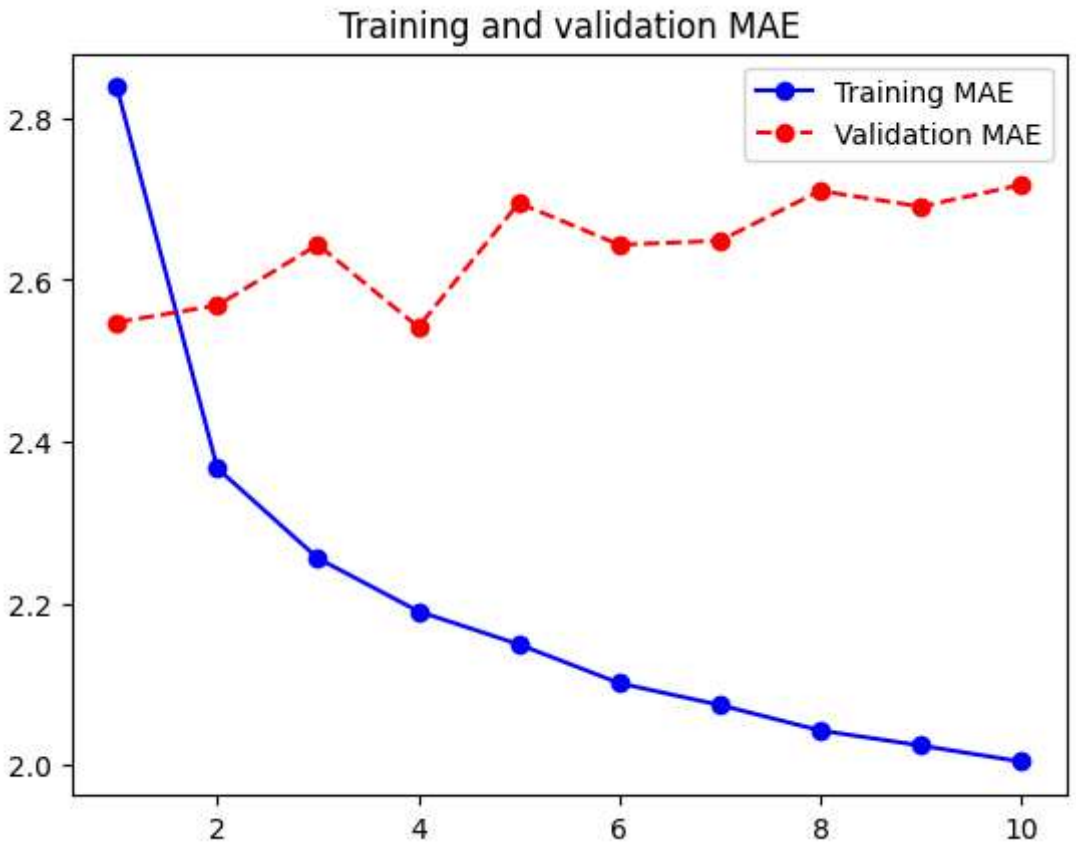
In [28]:
```
import keras
from keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1])) # raw_data.sha
x = layers.Flatten()(inputs)
x = layers.Dense(16, activation="relu")(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [ # Save the best performing model
    keras.callbacks.ModelCheckpoint("jena_dense.keras",
                                    save_best_only=True)
]
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
```

```
Epoch 1/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 23s 28ms/step - loss: 22.7451 - mae: 3.5425 - val_lo
ss: 10.4998 - val_mae: 2.5476
Epoch 2/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 24s 29ms/step - loss: 9.3293 - mae: 2.4044 - val_los
s: 10.6045 - val_mae: 2.5686
Epoch 3/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 27ms/step - loss: 8.3404 - mae: 2.2688 - val_los
s: 11.1636 - val_mae: 2.6437
Epoch 4/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 27ms/step - loss: 7.8500 - mae: 2.2045 - val_los
s: 10.4618 - val_mae: 2.5417
Epoch 5/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 27ms/step - loss: 7.4712 - mae: 2.1544 - val_los
s: 11.5782 - val_mae: 2.6952
Epoch 6/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 23s 28ms/step - loss: 7.1184 - mae: 2.1084 - val_los
s: 11.1533 - val_mae: 2.6435
Epoch 7/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 23s 28ms/step - loss: 6.8875 - mae: 2.0736 - val_los
s: 11.2127 - val_mae: 2.6485
Epoch 8/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 27ms/step - loss: 6.7222 - mae: 2.0493 - val_los
s: 11.6559 - val_mae: 2.7103
Epoch 9/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - loss: 6.6204 - mae: 2.0344 - val_los
s: 11.6560 - val_mae: 2.6907
Epoch 10/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - loss: 6.4467 - mae: 2.0071 - val_los
s: 11.7422 - val_mae: 2.7177
```

Plotting results

In [29]:
```python
import matplotlib.pyplot as plt

def plot_mae(history):
    loss = history.history["mae"]
    val_loss = history.history["val_mae"]
    epochs = range(1, len(loss) + 1)
    plt.figure()
    plt.plot(epochs, loss, "b-o", label="Training MAE")
    plt.plot(epochs, val_loss, "r--o", label="Validation MAE")
    plt.title("Training and validation MAE")
    plt.legend()
    plt.show()
```

In [30]:
```python
plot_mae(history)
```



Test results

```
In [32]: # Reload the best modle and evaluate it on the test data
         model = keras.models.load_model("jena_dense.keras")
         print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
405/405 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 11.2711 - mae: 2.6224
Test MAE: 2.62
```

- The results are close to the no-learning baseline.
- This goes to show the merit of having this baseline in the first place: it turns out to be not easy to outperform.
- Our common sense contains a lot of valuable information to which a machine learning model doesn't have access.

4.2.3. Time series data with convolutional networks

Let's try a 1D convolutional model

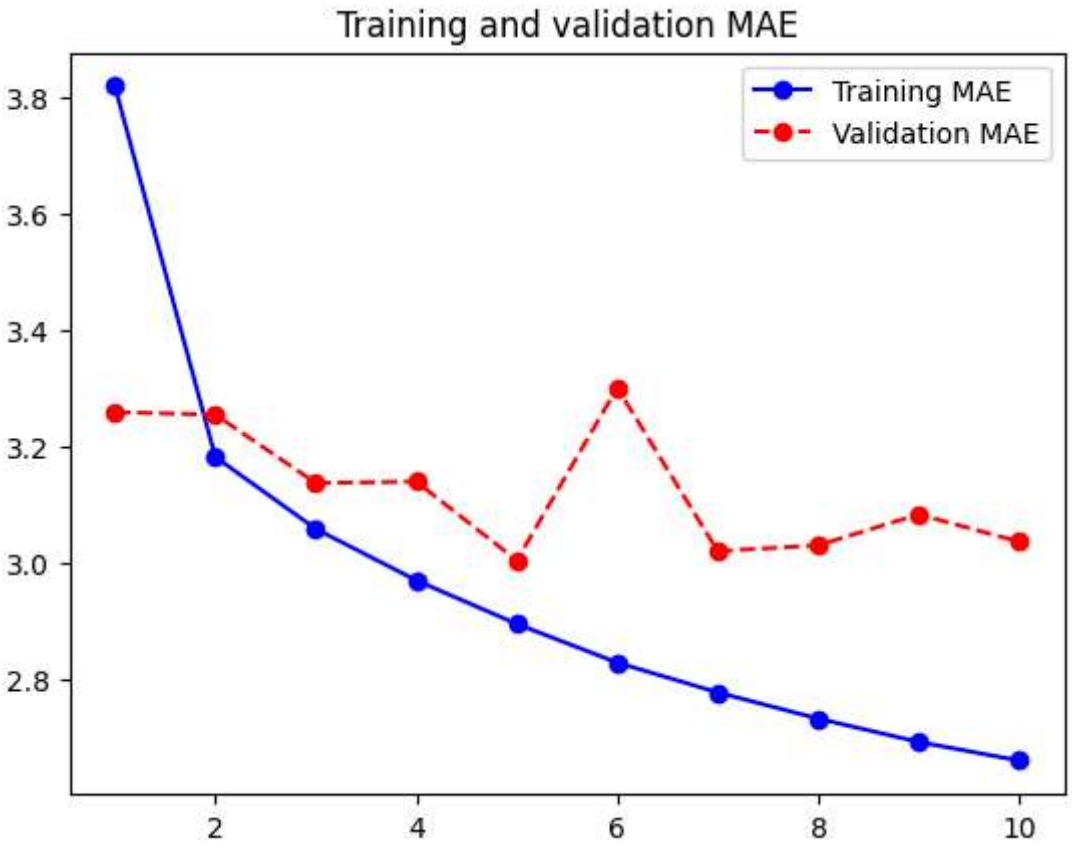
- We can build 1D convnets, strictly analogous to 2D convnets.
- They're a great fit for any sequence data that follows the translation invariance assumption (meaning that if you slide a window over the sequence, the content of the window should follow the same properties independently of the location of the window).
- Let's try one on our temperature-forecasting problem. We'll pick an initial window length (the size of our kernel) of 24, so that we look at 24 hours of data at a time

(one cycle). As we downsample the sequences (via MaxPooling1D layers), we'll reduce the window size accordingly:

In [33]:
```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Conv1D(8, 24, activation="relu")(inputs)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 12, activation="relu")(x)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 6, activation="relu")(x)
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_conv.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_conv.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
Epoch 1/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 39.6456 - mae: 4.7180 - val_lo
ss: 16.6397 - val_mae: 3.2583
Epoch 2/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 16.2388 - mae: 3.2042 - val_lo
ss: 17.1690 - val_mae: 3.2540
Epoch 3/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 14.9497 - mae: 3.0680 - val_lo
ss: 15.8411 - val_mae: 3.1366
Epoch 4/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 14.0463 - mae: 2.9724 - val_lo
ss: 15.8979 - val_mae: 3.1391
Epoch 5/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 13.3951 - mae: 2.9045 - val_lo
ss: 14.3044 - val_mae: 3.0037
Epoch 6/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 12.7795 - mae: 2.8335 - val_lo
ss: 17.1564 - val_mae: 3.3002
Epoch 7/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 12.3231 - mae: 2.7791 - val_lo
ss: 14.4061 - val_mae: 3.0196
Epoch 8/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 11.8810 - mae: 2.7321 - val_lo
ss: 14.7078 - val_mae: 3.0296
Epoch 9/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 11.5041 - mae: 2.6890 - val_lo
ss: 15.0174 - val_mae: 3.0828
Epoch 10/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 11.2155 - mae: 2.6568 - val_lo
ss: 14.7976 - val_mae: 3.0358
405/405 ━━━━━━━━━━━━━━━━━━━━ 6s 16ms/step - loss: 15.2124 - mae: 3.0945
Test MAE: 3.09
```

```
In [34]: plot_mae(history)
```



- As it turns out, this model performs even worse than the densely connected one, only achieving a validation MAE of about 2.9 degrees, far from the common-sense baseline.
- **Problems**:
  - Weather data doesn't quite respect the translation invariance assumption.
    - While the data does feature daily cycles, data from a morning follows different properties than data from an evening or from the middle of the night.
    - Weather data is only translation-invariant for a very specific timescale.
  - Order in our data matters.
    - The recent past is far more informative for predicting the next day's temperature than data from five days ago.
    - A 1D convnet is not able to leverage this fact. In particular, our max pooling and global average pooling layers are largely destroying order information.

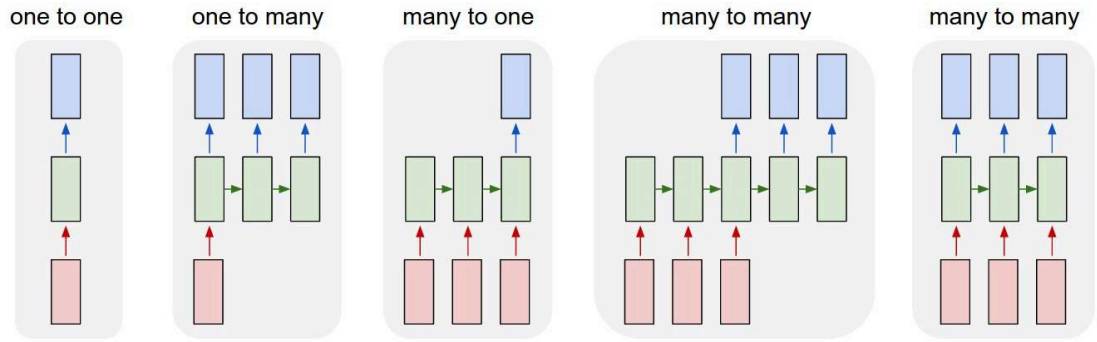
4.2.4. The need of recurrence

- **Dense NNs and CNNS have no memory**:
  - Each input shown to them is processed independently, with no state kept between inputs.

    - With such networks, in order to process a sequence or a temporal series of data points, you have to show the entire sequence to the network at once: turn it into a single data point.
- **Dense NNs and CNNS are too constrained**:
    - Simple Neural Networks and Convolutional Networks accept a fixed-sized vector as input (e.g. an image) and produce a fixed-sized vector as output (e.g. probabilities of different classes).
    - These models perform this mapping using a fixed amount of computational steps (e.g. the number of layers in the model).

4.3. Simple Recurrent Networks

- **Recurrent neural networks**
    - They process sequences by iterating through the sequence elements and maintaining a state that contains information relative to what it has seen so far.
    - They allow us to operate over sequences of vectors: Sequences in the input, the output, or in the most general case both.



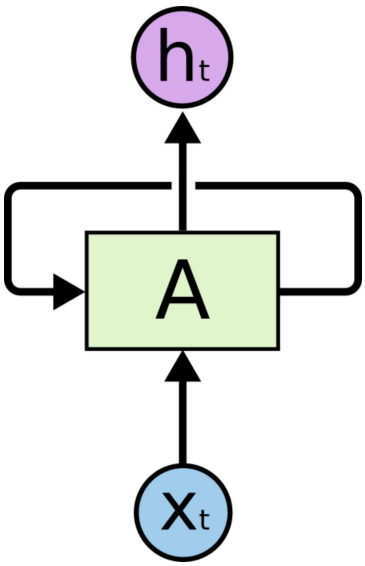
- **One to one**
    - The way of functioning of Simple NNs and CNNs.
    - From fixed-sized input to fixed-sized output.
    - Example:
        - *image classification.*
- **One to many**
    - Sequence output.
    - Example:
        - *Image captioning* takes an image and outputs a sentence of words.
- **Many to one**
    - Sequence input.
    - Examples:
        - *Sentiment analysis* where a given sentence is classified as expressing positive or negative sentiment.
        - *Time series prediction* where given a initial set of elements in a series we have to predict the next element.

- **Many to many**
  - Sequence input and sequence output.
  - Examples:
    - *Machine Translation*: an RNN reads a sentence in English and then outputs a sentence in French.
    - Time series prediction but we have to predict the next few elements.
- **Synced many to many**
  - Synced sequence input and output.
  - Example:
    - *Video classification* where we wish to label each frame of the video).

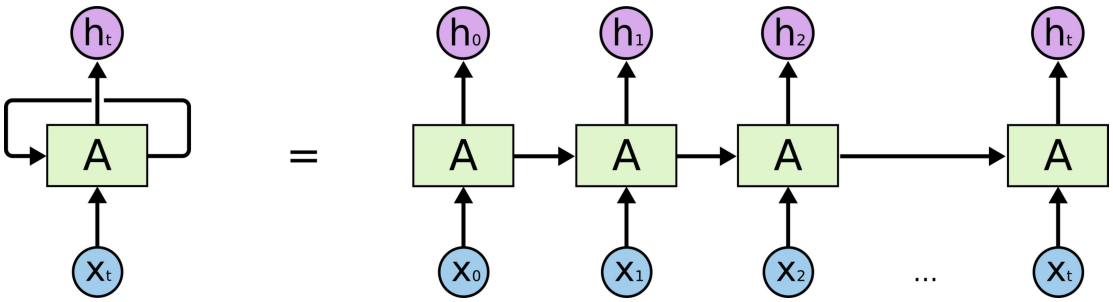
4.3.1. Recurrent connections

Graphical representation

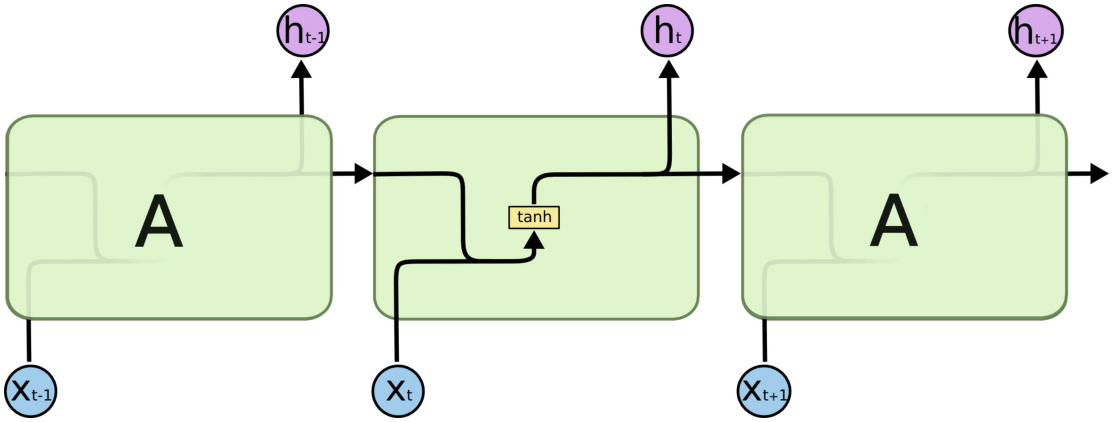
- Simple recurrent neural networks (also called classic or vanilla RNNs) have a hidden layer of neurons with a recurrent connection.
- In the diagram we can see a neural network A that accepts some input x_t and outputs a value h_t.
- A loop in A allows information to be passed to the same network in the next step of time.
- As we can see simple RNNs are networks with loops in them, allowing information to persist.

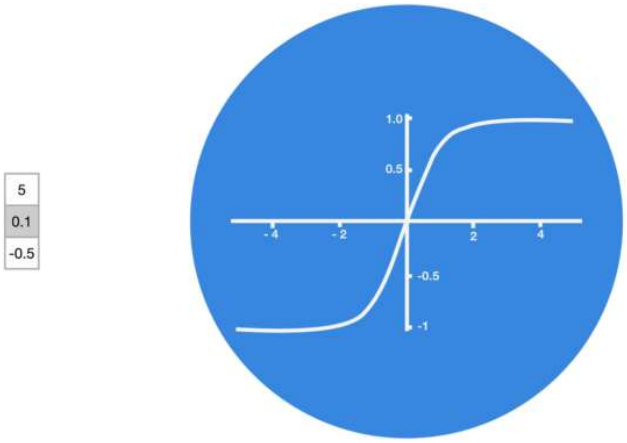


- We can unroll the RNN to see how it works over time.



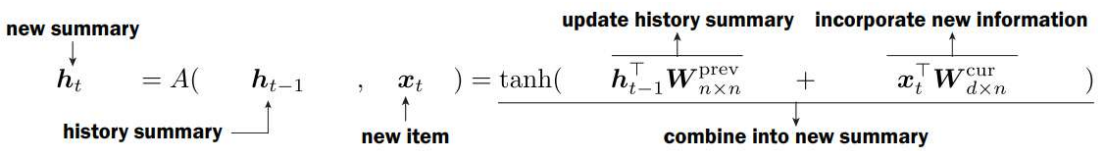
- Seeing this unrolled structure in more detail we can see that the output at time t (h_t) is the result of the input at that time (X_t) plus a hidden state that comes from the previous execution (the output at $t-1$). All of this passed through the activation function (`tanh`).





Mathematical representation

- Mathematically the recurrent connection can be described as:

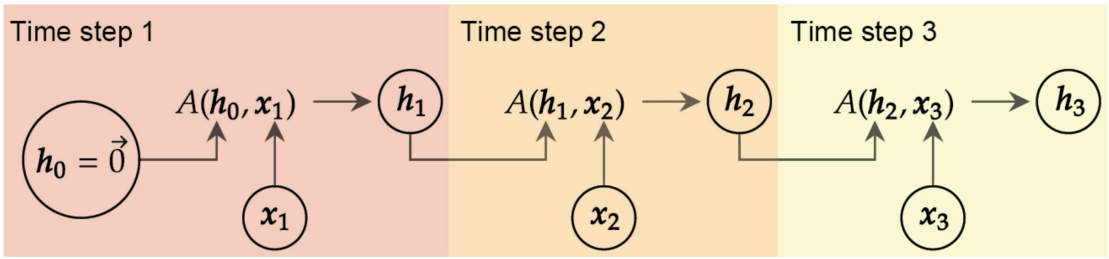


$$\boldsymbol{h}_t = A(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t) = \tanh(\boldsymbol{h}_{t-1}^{\top}\boldsymbol{W}_{n\times n}^{\text{prev}} + \boldsymbol{x}_t^{\top}\boldsymbol{W}_{d\times n}^{\text{cur}})$$

- Being:
  - d: The dimension of the input in a given timestep.
  - n: The dimension of the output in a given timestep.
  - $W_{d\times n}^{cur}$: The weights for the current input

    - $W_{n \times n}^{prev}$: The weights for the recurrent connection (previous output)
- We can see that both the input connection and the recurrent connection have trainable weights.

- Having defined how the neural network A works we can unroll it showing the different timesteps.
- The initial hidden state is set to a vector of all zero values.



Code representation

- Pseudocode

```
state_t = 0
for input_t in input_sequence:
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    state_t = output_t
```

In [35]:
```
import numpy as np

timesteps = 100       # Number of timesteps in the input sequence. In Jena times
input_features = 32   # Dimensionality of the input feature space. In Jena input
output_features = 64  # Dimensionality of the output feature space. In Jena outp

inputs = np.random.random((timesteps, input_features)) # Input data: random nois
state_t = np.zeros((output_features,)) # Initial state: an all-zero vector

W = np.random.random((output_features, input_features))   # Input weights (initi
U = np.random.random((output_features, output_features))  # Recurrent weights (i
b = np.random.random((output_features,))                  # Bias (initialized at

successive_outputs = []

for input_t in inputs: # Traversing the different timesteps, input_t is a vector
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b) # Combines t
    successive_outputs.append(output_t) # Stores this output in a list
    state_t = output_t # Updates the state of the network for the next timestep

final_output_sequence = np.stack(successive_outputs, axis=0) # The final output
```

- In this example, the final output is a rank-2 tensor of shape `(timesteps, output_features)`, where each timestep is the output of the loop at time `t`.
- Each timestep `t` in the output tensor contains information about timesteps `0` to `t` in the input sequence—about the entire past.

- For this reason, in many cases, you don't need this full sequence of outputs; you just need the last output (`output_t` at the end of the loop), because it already contains information about the entire sequence.

4.3.2. Recurrent layers in Keras

Shape of the inputs

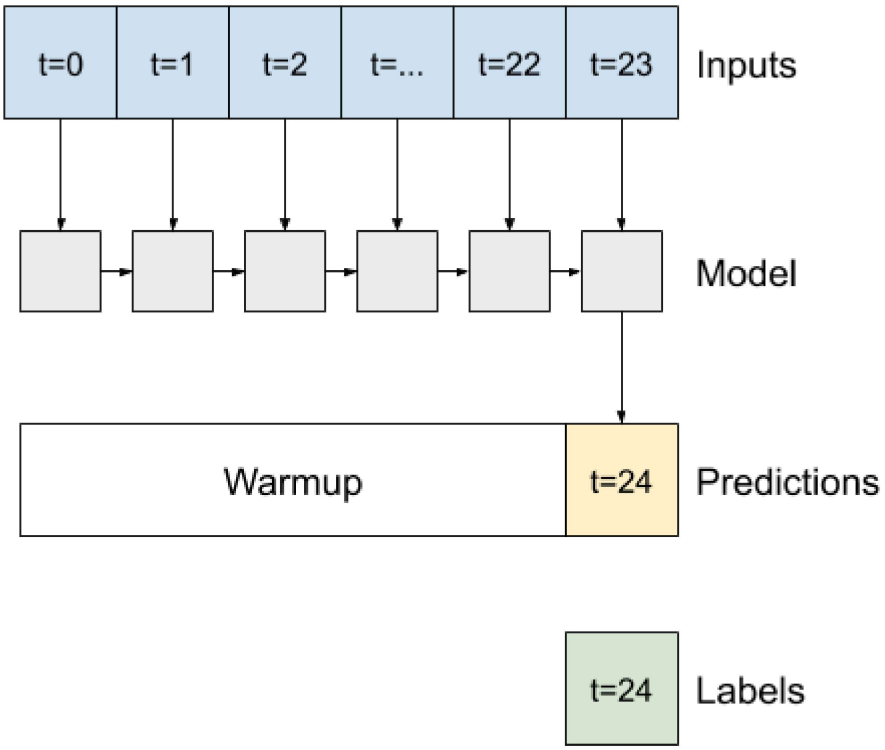
- The layer for implementing simple RNNs in Keras is `SimpleRNN`.
- It takes inputs of shape (`batch_size` , `timesteps` , `input_features`).
- When specifying the shape argument of the initial `Input()` , note that you can set the timesteps entry to `None` , which enables your network to process sequences of arbitrary length. This is especially useful if your model is meant to process sequences of variable length.
- However, if all of your sequences have the same length, it is recommended specifying a complete input shape, since it enables `model.summary()` to display output length information and it can unlock some performance optimizations.
- More information: https://keras.io/api/layers/recurrent_layers/simple_rnn/

Main arguments of `SimpleRNN` constructor:

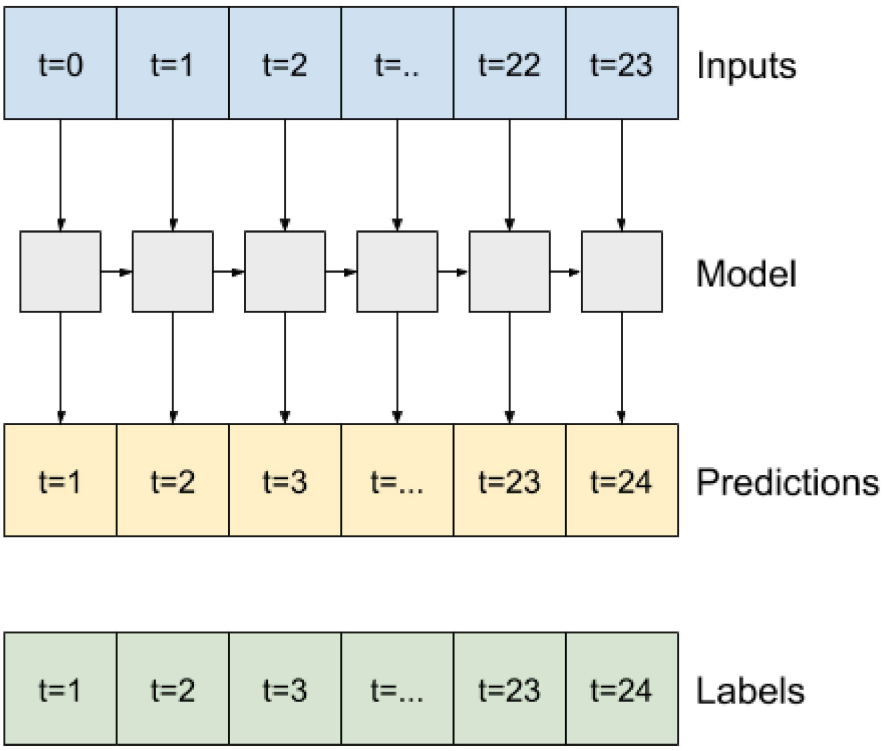
- `units` : Positive integer, dimensionality of the output space.
- `activation` : Activation function to use. Default: hyperbolic tangent (`tanh`).
- `dropout` : Float between 0 and 1. Fraction of the units to drop for the linear transformation of the inputs. Default: `0` .
- `recurrent_dropout` : Float between 0 and 1. Fraction of the units to drop for the linear transformation of the recurrent state. Default: `0` .
- `return_sequences` : Boolean. Whether to return the last output in the output sequence, or the full sequence. Default: `False` .

`return_sequences` argument

- If `False` , the default, the layer only returns the output of the final time step, giving the model time to warm up its internal state before making a single prediction.

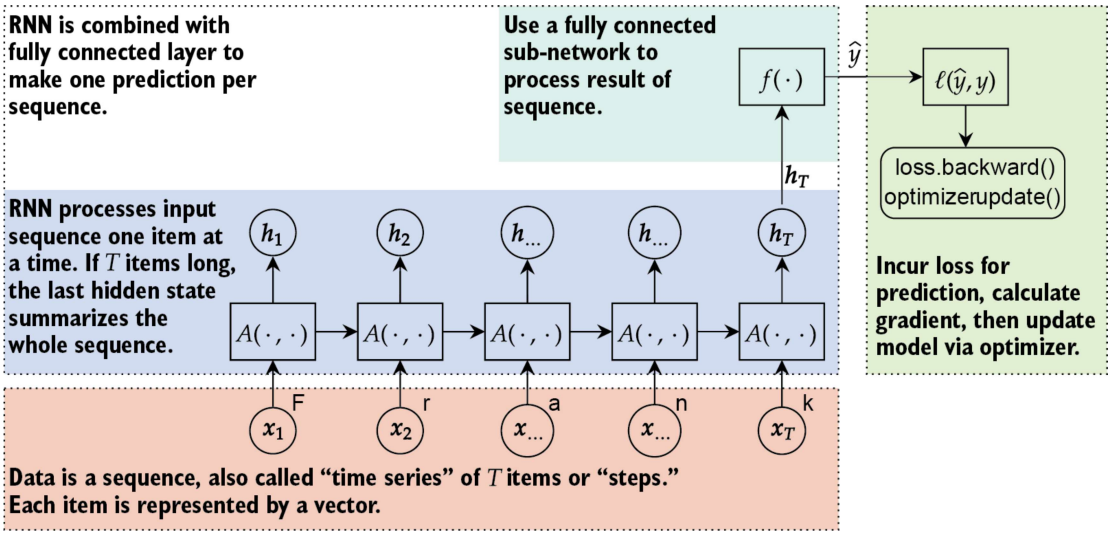


- If `True`, the layer returns an output for each input. This is useful for:
  - Stacking RNN layers.
  - Training a model on multiple time steps simultaneously.



4.3.3. A simple RNN for sequence data

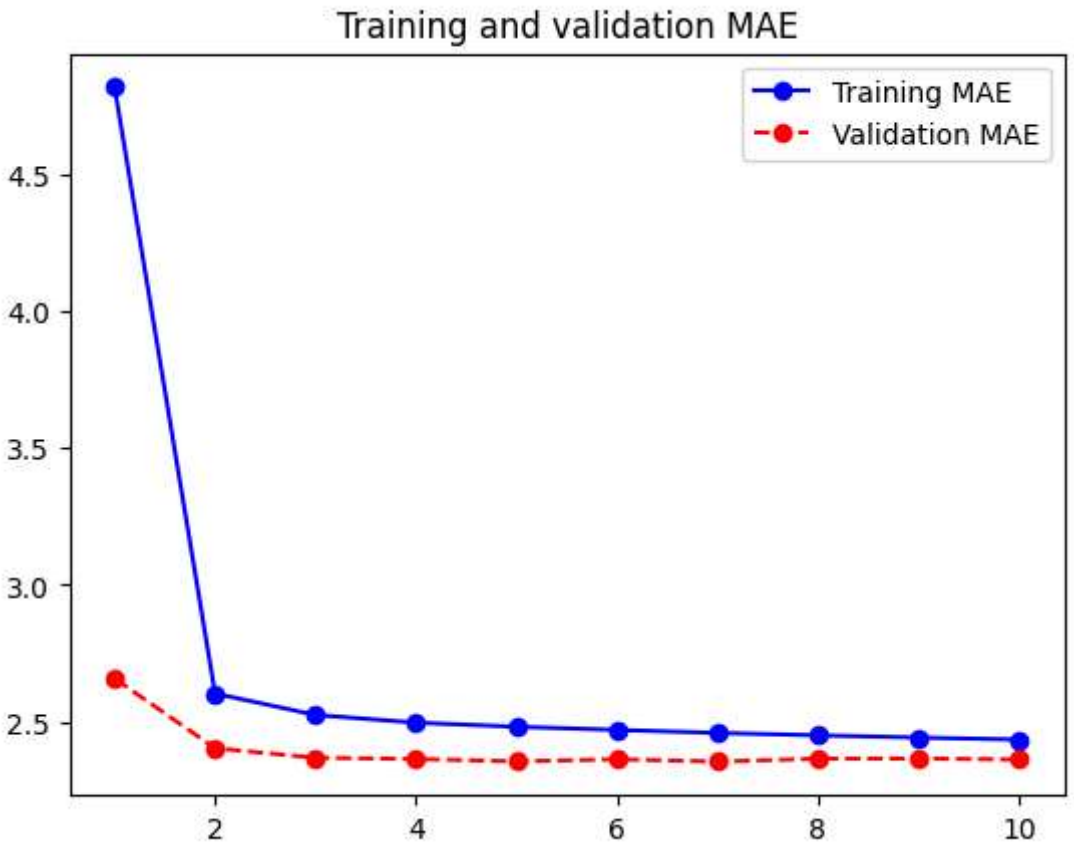
- Let's build a simple RNN for our Jena dataset with the following characteristics:
  - The size of our sequence is 120 (`sequence_length = 120`).
  - The input size is `(120, 14)`
  - The output size of the single RNN is set to `16` .
  - `return_sequences` is left at its default value (`False`).
  - These `16` values obtained at the end of sequence processing are used as input to a dense network:
    - It has only one output neuron that represents the temperature prediction.
    - It has no activation function since it is a regression problem.
- The operating scheme of the network would be as follows:



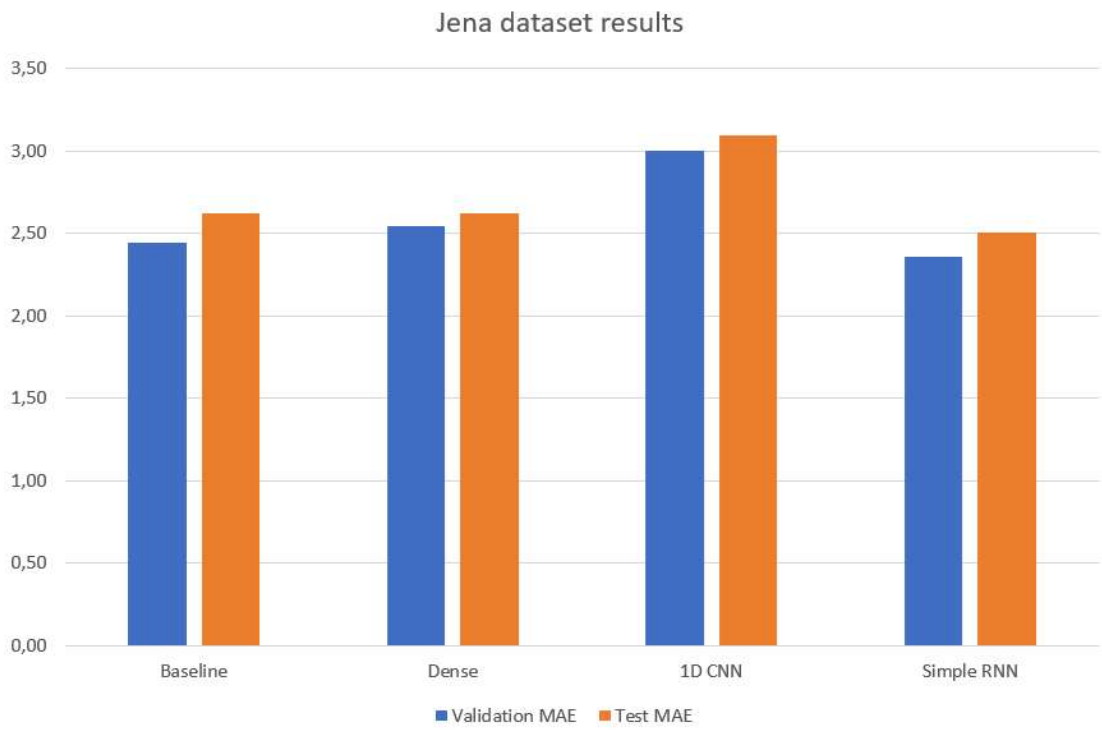
In [36]:
```python
import keras
from keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1])) # (120, 14)
x = layers.SimpleRNN(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_rnn.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_rnn.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
Epoch 1/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 220s 269ms/step - loss: 73.5045 - mae: 6.7154 - val_
loss: 12.4568 - val_mae: 2.6579
Epoch 2/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 214s 261ms/step - loss: 11.8934 - mae: 2.6590 - val_
loss: 9.5754 - val_mae: 2.4031
Epoch 3/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 214s 261ms/step - loss: 10.5113 - mae: 2.5280 - val_
loss: 9.2737 - val_mae: 2.3683
Epoch 4/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 213s 260ms/step - loss: 10.2332 - mae: 2.4967 - val_
loss: 9.2161 - val_mae: 2.3639
Epoch 5/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 214s 262ms/step - loss: 10.1015 - mae: 2.4808 - val_
loss: 9.1540 - val_mae: 2.3549
Epoch 6/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 213s 260ms/step - loss: 10.0016 - mae: 2.4668 - val_
loss: 9.2010 - val_mae: 2.3629
Epoch 7/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 211s 258ms/step - loss: 9.9232 - mae: 2.4562 - val_l
oss: 9.1678 - val_mae: 2.3543
Epoch 8/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 229s 280ms/step - loss: 9.8595 - mae: 2.4484 - val_l
oss: 9.2250 - val_mae: 2.3650
Epoch 9/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 247s 302ms/step - loss: 9.7802 - mae: 2.4409 - val_l
oss: 9.2452 - val_mae: 2.3657
Epoch 10/10
819/819 ━━━━━━━━━━━━━━━━━━━━ 207s 253ms/step - loss: 9.7133 - mae: 2.4320 - val_l
oss: 9.2325 - val_mae: 2.3618
405/405 ━━━━━━━━━━━━━━━━━━━━ 52s 127ms/step - loss: 10.4724 - mae: 2.5010
Test MAE: 2.50
```

In [37]:
```
plot_mae(history)
```



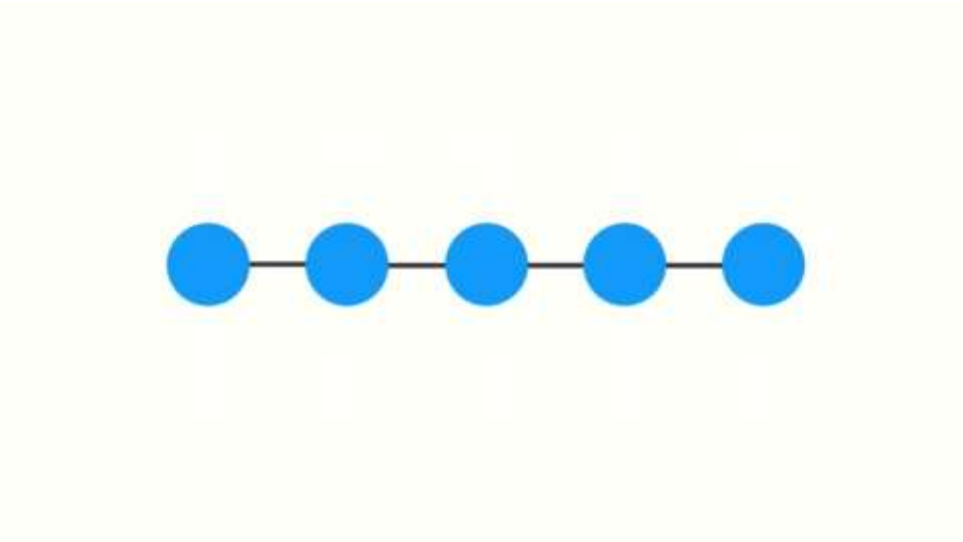
- We achieve a validation MAE as low as 2.33 degrees and a test MAE of 2.47 degrees.
- We beat the common-sense baseline.



4.3.4. Problems with simple recurrent networks

- The problem with simple RNNs is that they are unable to identify long-term dependencies over time.

- Long-term dependencies, in this case, can be defined as a desired output at time `T` which is dependent on inputs from times `t` less than `T`.
- RNNs were able to learn short-term dependencies with gradient learning based methods, but had difficulty with long-term dependencies.
- Bengio and Hochreiter have both noted that the problem of "*vanishing gradients*" is the reason why a network trained by gradient-descent methods is unable to distinguish a relationship between target outputs and inputs that occur at a much earlier time.
- As a system robustly latches information, the fraction of the gradient due to information t-steps in the past approaches zero as t becomes large.



- This problem is similar to the "vanishing gradients" that occurs during backpropagation.